Master Thesis Technical Report: Phase I Dataset and Phase II C1 Stability Analysis

Ali Bakhshayesh

May 8, 2026

Abstract

This report consolidates Phase I of the thesis and begins Phase II comparative evaluation. The Phase I dataset contains ten open-source repositories, each frozen at eight stable release tags and seven consecutive release windows. The report documents repository and version-window curation, metric extraction for M1–M4, cleaning and validation, bot-sensitivity handling, and duration-normalized outputs for uneven release windows. It then reports the first Month 3 analysis step: C1 stability across release windows and projects. No proof-of-concept (PoC) metric is selected here; that decision remains deferred until the remaining comparison criteria and feasibility scoring are complete.

Contents

List of Tables

List of Figures

1 Abbreviations and Notation

Term	Meaning in this report
PoC	Proof-of-concept smart-contract implementation (selected after Phase II only).
DESTINO	Target evidence-based developer-reputation framework used to define the smart-contract evidence boundary.
LOC/SLOC	Total/source line counts used as snapshot-size indicators (M1).
Churn	Added plus deleted lines in a window (M2).
Cadence	Activity continuity over time (commits, active days, and gaps) (M3).
Ownership concentration	Distribution of developer contribution shares (M4).
Window	Consecutive tag interval ($T_k \rightarrow T_{k+1}$) in chronological order.
All-account	Metric population including both human and automated Git author identities.
Human-only	Metric population excluding rows flagged as automated accounts via `is_bot`.
Pilot cohort	The first four repositories used to validate the workflow before expansion.
Extended cohort	Six additional repositories added to test the pipeline across more ecosystems.
Centralized Phase I dataset	Final 10-repository dataset used as the Phase I evidence base.

Table 1: Abbreviations and working notation used throughout the report.

2 Introduction, Scope, and Research Roadmap

This thesis studies software-quality indicators as evidence for developer reputation systems. The objective of the current phase is methodological: construct a reproducible multi-project dataset, extract candidate metrics with transparent rules, and produce analysis-ready outputs for comparative evaluation. No proof-of-concept (PoC) metric is selected in this phase; metric selection is deferred to Phase II.

Research Questions

- **RQ1 (Stability):** Which candidate indicators are stable across versions and comparable across projects?
- **RQ2 (Interpretability and gaming):** Which indicators are understandable and robust against manipulation?
- **RQ3 (Smart-contract feasibility):** Which indicator is the strongest PoC candidate under implementation constraints?

Document Structure

The report is organized as follows:

- Sections 3–7 define candidate metrics, comparison criteria, DESTINO alignment, feasibility scoring, and the future PoC integration boundary.
- Sections 8–11 document dataset curation, extraction, cleaning, and descriptive outputs for the centralized 10-repository Phase I dataset.
- Sections 14–16 provide reproducibility and validity considerations.
- Section 18 defines the comparative evaluation plan and decision gate for PoC metric selection.

Contributions

- A shortlist of candidate metrics and operational comparison criteria.
- A fully reproducible 10-repository Phase I dataset with frozen repositories, tags, and consecutive windows.
- Cleaned, validated metric datasets (developer-window and window totals) and ownership/concentration statistics.
- Explicit bot-sensitivity and short-window sensitivity outputs, so automation and release-cadence artifacts are visible rather than hidden.
- A detailed smart-contract feasibility rubric to support later PoC metric selection.
- A DESTINO-aligned evidence model showing how Phase I outputs map to a future metric-agnostic PoC.

3 Candidate Metrics (Shortlist)

We compare a minimal set of candidate indicator families that are (i) computable from versioned artifacts and Git history, (ii) interpretable and comparable across heterogeneous projects, and (iii) compatible with hash-anchored evidence and compact on-chain storage. The shortlist is also informed by existing smart-contract metric and complexity literature [10, 8]. The comparative study evaluates all candidates; the PoC metric is selected only after evaluation.

Candidate	Usage in this thesis	Why chosen (one sentence)
M1 Snapshot size (LOC/SLOC)	Per-tag size baseline for normalization and context.	Simple, interpretable baseline; efficient to compute, store, and audit.
M2 Patch complexity proxy (churn)	Added/deleted lines per release window and developer.	Minimal proxy for change magnitude; aggregation-friendly and auditable.
M3 Activity cadence	Commits/activity-days per window; inactivity gaps.	Captures continuity over time without requiring platform-specific PR data.
M4 Ownership / concentration	Developer shares of churn/commits; top-1/top-3 and Gini concentration.	Separates core maintainers from occasional contributors; supports stability analysis.
M5 Maintenance responsiveness (design extension)	Documented schema/logic; not implemented in baseline PoC.	Included for completeness, but deferred to keep baseline PoC scope controlled.

Table 2: Candidate shortlist. All metrics are computed and compared before selecting a single PoC metric.

4 Methodology: Comparison Criteria

We compare candidates using four criteria: stability, interpretability, resistance to gaming, and long-term suitability. The comparative phase (Phase II) applies these criteria uniformly to M1–M5 before any PoC metric decision.

4.1 Evaluation unit and normalization policy

The primary analysis unit is `(project_id, window_id, developer)` for developer-level metrics and `(project_id, window_id)` for aggregate metrics. Because window durations and project sizes vary, the pipeline now produces both raw and normalized values:

- per-day normalization for cadence-sensitive measures;
- per-KLOC normalization where size effects can dominate;
- within-project shares (ownership) to control for absolute scale.

Windows shorter than seven days are retained for auditability but flagged as `sensitivity_only`; Phase II reports both full-dataset results and robust results excluding those very short windows when duration-sensitive metrics are evaluated.

Automation is handled as an explicit population rule. Rows flagged with `is_bot` remain in the clean all-account tables for auditability and project-activity totals. However, developer-behavior metrics use human-only outputs as the primary population. In the centralized Phase I dataset, M3 cadence and M4 ownership/concentration are evaluated from `out/metrics_window_dev_human_all.csv` and `out/ownership_summary_human_all.csv`. Bot summaries are reported as sensitivity evidence rather than silently mixed into maintainer-behavior claims.

4.2 C1 Stability

Stability is evaluated across consecutive version windows. We measure volatility (e.g., dispersion across windows) and rank stability (e.g., correlation of developer rankings between adjacent windows). Cross-project comparison is performed after simple normalizations (e.g., per day or per KLOC) to reduce artifacts from project size and release cadence. Planned outputs include dispersion statistics, adjacent-window rank correlations, and sensitivity checks on timestamp semantics.

4.3 C2 Interpretability

Interpretability is assessed via semantic clarity (plain meaning), unit clarity (counts/shares/time), and normalization clarity. Metrics that require complex tooling or ambiguous definitions receive lower interpretability.

4.4 C3 Resistance to gaming

We analyze attack surfaces and manipulation cost per metric. Since single metrics can be misleading, we avoid strong causal claims and interpret gaming risks comparatively (e.g., commit-splitting inflates frequency but becomes detectable when churn remains low). The threat model follows common public-ledger constraints and manipulation tradeoffs discussed in blockchain and smart-contract studies [1, 10, 8].

4.5 C4 Long-term suitability

We assess whether each metric remains informative over long histories, across lifecycle phases, and whether it supports incremental updates without full recomputation.

4.6 Criterion-to-output mapping

Criterion	Planned Phase II evidence	Primary data source(s)
C1 Stability	Window-level volatility, adjacent-window rank correlation, robustness to normalization choices.	`out/metrics_window_dev_human_normalized_all.csv`, `out/metrics_window_totals_human_normalized_all.csv`, `out/window_duration_flags_all.csv`
C2 Interpretability	Structured rubric (semantic clarity, unit clarity, normalization clarity, tooling transparency).	Metric definitions plus extracted fields in `out/` artifacts
C3 Gaming resistance	Attack-surface matrix and manipulation detectability checks under multi-metric reading.	`out/metrics_window_dev_human_all.csv`, `out/ownership_dev_shares_human_all.csv`, `out/bot_summary_by_project_all.csv`
C4 Long-term suitability	Informativeness over time, lifecycle robustness, and incremental update feasibility.	`out/loc_per_tag_all.csv`, `data/windows_all.csv`, feasibility rubric dimensions

Table 3: Operational mapping from methodology criteria to evidence sources.

4.7 Decision protocol (predefined, not executed in Phase I)

The decision protocol is fixed before Phase II analysis:

1. compute comparative evidence for all candidates under C1–C4;
2. score smart-contract feasibility using the rubric in Section 6.1;
3. eliminate any candidate that violates feasibility exclusion criteria;
4. select one PoC metric with the best overall tradeoff between empirical behavior and feasibility.

No step of this decision protocol is executed in Phase I.

5 DESTINO Alignment and Evidence Boundary

DESTINO is the reference deployment context for the thesis: software-quality evidence should support developer reputation claims through signed submissions, immutable records, and replayable verification. In this report, DESTINO is not treated as a reason to move all computation on-chain. Instead, it defines the evidence boundary that every candidate metric must satisfy before a proof-of-concept metric can be selected.

The working DESTINO-aligned evidence model has four parts:

- a developer-facing submission containing project/archive metadata, software-type tags, storage-persistence metadata, and a cryptographic commitment to the submitted artifact;
- an off-chain measurement pipeline that retrieves the committed artifact, verifies integrity, and computes the selected metric;
- compact on-chain records containing the submission reference, metric value, and evidence hash;
- public auditability through replay: retrieve the same artifact, rerun the declared script/configuration, and compare the result with the committed metric record.

This boundary keeps expensive repository traversal, Git history analysis, and tabular validation off-chain, while the smart-contract layer stores only bounded metadata, commitments, and metric outputs. This matches common blockchain engineering constraints: large artifacts and unbounded loops are poor on-chain candidates, while hash commitments and compact records are auditable and storage-bounded [1].

5.1 Implications for the Current Report

The Phase I report is therefore not only a data-extraction report. It prepares the exact evidence artifacts that a later DESTINO PoC would need. The alignment is summarized in Table 4.

DESTINO need	Phase I artifact or rule	Why it matters
Stable artifact identity	`data/dataset_freeze_all.csv`, `data/versions_all.csv`, frozen commit hashes	Metric evidence can be tied to explicit projects, tags, and repository states.
Replayable measurement interval	`data/windows_all.csv`	Each metric value has a deterministic release-window boundary.
Compact metric evidence	`out/loc_per_tag_all.csv`, clean metric tables, normalized tables, human-only M3/M4 tables	The future contract can store selected values without storing raw repositories or diffs.
Audit and validation evidence	`logs/*.log`, cleaning checks, duration flags, bot-sensitivity summaries	A verifier can inspect how raw extraction became analysis-ready evidence.
Smart-contract selection discipline	Criteria C1–C4 plus the feasibility rubric	The selected PoC metric must be empirically useful and feasible under bounded contract execution.

Table 4: Alignment between DESTINO evidence requirements and current Phase I artifacts.

Two current design decisions are especially important for DESTINO alignment. First, duration-normalized outputs and `sensitivity_only` flags prevent release cadence artifacts from being silently recorded as comparable metric evidence. Second, human-only M3/M4 outputs prevent automated accounts from being interpreted as human maintainer behavior. The all-account tables remain useful as an audit trail, but the primary developer-behavior evidence is explicitly human-only.

No PoC metric is selected in Phase I. DESTINO alignment at this stage means the pipeline produces evidence that can later be committed, audited, and scored; it does not mean that any current metric has already passed the comparative and feasibility gate.

6 Extended Feasibility Rubric (Detailed)

6.1 Smart-Contract Feasibility Rubric

6.1.1 Why feasibility must be a first-class criterion

This thesis evaluates multiple candidate metrics for evidence-based developer reputation. However, the project explicitly requires that feasibility as a smart contract is considered as a selection criterion and that a proof-of-concept (PoC) is implemented for one chosen metric. The project scope further constrains the design by prescribing an evidence workflow where developers submit a signed form containing a link and a hash of an archive; the smart contract verifies the submission, retrieves and verifies the archive, computes metrics, writes to the ledger the form, metrics, tags, and a storage-persistence attribute, and signals a sibling search/index system for each new datapoint. Therefore, feasibility here is not a generic "blockchain compatibility" notion, but the practical ability to implement (or safely approximate) the required pipeline under smart-contract constraints while preserving auditability and integrity.

In the blockchain literature, feasibility is shaped by system constraints that include (i) performance limits and throughput constraints, (ii) storage growth and distribution overhead, and (iii) challenges in connecting on-chain logic to external services and data (oracle-style issues). These constraints make heavy computation and large state growth poor candidates for on-chain implementation, often motivating hybrid architectures in which bulk data remains off-chain and the chain stores commitments and compact metadata. Smart-contract ecosystems add further constraints: execution is costed (gas), code and execution patterns are shaped by platform limits, and determinism is required. Empirical work on Solidity metrics shows that many metric distributions are truncated and exist in restricted ranges due to platform constraints,

which supports treating feasibility as a distinct and non-trivial selection criterion rather than assuming traditional software metrics transfer unchanged [1, 10, 8].

6.1.2 Definition: smart-contract feasibility

We define **smart-contract feasibility** of a candidate metric as:

> The ability to compute, verify, and store a metric (or its verified output) within the defined evidence workflow with bounded on-chain computation and storage, minimal reliance on unverifiable external inputs, strong auditability (re-checking from archived evidence), and explicit binding to developer identity/provenance.

This definition is intentionally *architecture-aware*. A candidate can be feasible in one of two ways:

- **On-chain computable feasibility:** the metric can be computed directly in the contract with small, bounded logic.
- **Hybrid feasibility (recommended for PoC):** the metric is computed off-chain from a verified artifact (hash+link), and the contract stores only compact metric values plus hash commitments to evidence/inputs to enable auditability.

The rubric below evaluates feasibility in a way that allows both patterns, while explicitly penalizing designs that require unverifiable oracles, frequent high-throughput writes, or large on-chain state.

6.1.3 Rubric structure and scoring

Each metric receives a score on 10 dimensions, each scored in $\{0, 1, 2\}$ (0 = poor feasibility, 2 = strong feasibility). The rubric is used **only after** the multi-metric comparative analysis, to select the single PoC metric.

How to use the rubric (procedure).

1. For each candidate metric M_i, write a one-paragraph "implementation story": what are inputs, what is computed, what is stored, how often updated, who attests.
2. Score each rubric dimension using the guidance below.
3. Compute a total feasibility score and also record *exclusion conditions* (dimensions where score = 0 implies unacceptable risk for the PoC).
4. Select the PoC metric as the best trade-off between (i) empirical performance under C1–C4 and (ii) feasibility score, with tie-breaking favoring determinism, auditability, and bounded cost.

6.1.4 Feasibility dimensions (detailed)

Dimension	0–2	Scoring rule (detailed)
D1. Determinism of computation	0–2	**2:** metric computation is deterministic given the archived artifact and explicit parameters (window definition, tool version). **1:** deterministic only under additional assumptions (toolchain pinned; minor nondeterminism mitigated). **0:** depends on nondeterministic factors or ambiguous extraction (e.g., floating-point randomness, unstable parsing).
D2. Bounded on-chain computation	0–2	**2:** on-chain logic is constant-time or small bounded checks (signature format check, hash equality, record writes). **1:** moderate bounded computation per update (limited loops over small arrays). **0:** requires heavy analysis on-chain (AST parsing, full-history scanning) or unbounded loops.
D3. Bounded on-chain storage growth	0–2	**2:** store only compact values + commitments (hashes) + minimal metadata. **1:** store moderate aggregates (rolling sums, bounded windows). **0:** requires storing large histories/artifacts or per-commit detailed logs on-chain.
D4. Update frequency feasibility	0–2	**2:** metric can be updated per release or per time window (low write rate). **1:** requires medium-frequency updates (e.g., per PR/issue) but can be batched. **0:** requires high-frequency updates (per commit/event stream) and becomes throughput-limited.
D5. External data/oracle dependence	0–2	**2:** inputs are self-contained in the archived artifact + signed form, verifiable by hash commitments. **1:** uses off-chain data but can be posted as signed attestations with clear trust model. **0:** requires unverifiable external services or subjective judgments (strong oracle problem).

Table 5: Feasibility dimensions with detailed scoring rules (D1–D5).

Dimension	0–2	Scoring rule (detailed)
D6. Auditability and rechecking	0–2	**2:** third parties can recompute metric from archive retrieved via link and verified by on-chain hash; measurement report is committed. **1:** reproducible with effort (tooling dependencies; partial evidence). **0:** cannot be reliably reproduced or verified from public evidence.
D7. Identity binding and provenance	0–2	**2:** metric evidence is bound to developer identity via signed submissions (and the same identity used consistently across submissions). **1:** partial binding (mapping from commit authors; weaker identity guarantees). **0:** attribution cannot be trusted or can be trivially spoofed.
D8. Integrity of artifact linkage	0–2	**2:** strong commitment: archive hash stored on-chain and verified against retrieved archive; versioning explicit. **1:** link is stable but hash commitment incomplete or relies on weak assumptions. **0:** artifact cannot be pinned; link rot or mutable artifact breaks verification.
D9. Metric-value representation (range + precision)	0–2	**2:** metric stored as small integers or fixed-point with clear bounds; low risk of overflow/precision issues. **1:** requires careful normalization/scale but feasible (e.g., store numerator/denominator separately). **0:** requires floating point or complex statistics not representable safely on-chain.
D10. Adversarial resilience in a public ledger context	0–2	**2:** metric is hard/expensive to inflate without leaving detectable traces, especially with hash-anchored evidence. **1:** partially gameable but mitigations exist (normalization, batching, cross-check with other metrics in study). **0:** trivially gameable with low-cost manipulations (e.g., verbosity inflation) and weak detectability.

Table 6: Feasibility dimensions with detailed scoring rules (D6–D10).

6.1.5 Evidence checklist per dimension (what you must document)

To keep feasibility rigorous and evidence-based, each dimension must be supported by explicit evidence:

- **D1 Determinism:** specify exact inputs (archive hash, link, version id), the window definition, and (for off-chain computation) pin the tool version and configuration; describe how the PoC ensures the same artifact yields the same output.
- **D2/D3 Cost bounds:** list the contract operations required per datapoint (comparisons, storage writes); argue boundedness.
- **D4 Update frequency:** define update cadence (per release/time window). Explain why per-commit updates are avoided.
- **D5 Oracle dependence:** list any off-chain sources beyond the archive; if any exist, state whether they are hash-anchored or signed attestations.

- **D6 Auditability:** define the *re-check protocol*: retrieve archive, verify hash, recompute metric, compare with ledger value.
- **D7/D8 Provenance and linkage:** describe how the signed form + hash commitment prevents substitution of artifacts.
- **D9 Representation:** define encoding (int, fixed-point, store numerator/denominator); describe overflow handling and normalization.
- **D10 Adversarial model:** list low-cost manipulations and whether the system can detect them via evidence consistency.

6.1.6 Feasibility Exclusion Conditions

Even if a metric is strong empirically, it is a weak PoC candidate if it triggers exclusion conditions:

- **Exclusion Condition A:** score 0 on D1 (non-deterministic), D6 (non-auditable), or D8 (artifact cannot be pinned).
- **Exclusion Condition B:** requires high-frequency updates (D4=0) that exceed realistic ledger throughput.
- **Exclusion Condition C:** depends on unverifiable external facts (D5=0) without a signed-attestation model.

6.1.7 How this rubric maps to the project evidence workflow

The project workflow specifies a procedure where a smart contract: (i) checks the signature in the form, (ii) retrieves the archive from the link, (iii) checks signature and hash match, (iv) measures quality metrics, (v) writes form + metrics + tags + storage-persistence reference, and (vi) signals a sibling indexing system. The rubric operationalizes this in dimensions D1, D6, D7, and D8 (determinism, auditability, identity binding, artifact linkage), and constrains practical implementation via D2–D5 and D9 (bounded compute/storage, update frequency, oracle dependence, representation).

6.1.8 How the rubric is applied to the five thesis candidates (template only)

Table 7 is used after Phase II to score each candidate. Scores are not assigned in the current reporting phase.

Metric	D1	D2	D3	D4	D5	D6	D7	D8	D9	D10
M1 Snapshot size (LOC/SLOC)	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
M2 Patch complexity (churn)	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
M3 Activity cadence	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
M4 Ownership/concentration	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
M5 Maintenance responsiveness (design ext.)	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A

Table 7: Feasibility rubric status table. All scores are deferred to Phase II comparative evaluation.

7 Future DESTINO PoC Integration Blueprint

This section records the intended integration shape for the future proof-of-concept without choosing the PoC metric. Its purpose is to keep Phase I connected to the deployment target while preserving the decision rule: empirical comparison and feasibility scoring must happen before implementation.

7.1 Design Objective and Boundary

The future PoC should support evidence-based developer reputation records that are traceable to hash-pinned software artifacts. The contract layer should not compute repository metrics directly. Repository traversal, Git history analysis, duration normalization, bot handling, and ownership calculations remain off-chain because they require file-system access, Git tooling, and potentially unbounded iteration. The contract stores the result and the commitments needed to audit it.

The boundary is:

- **Off-chain:** artifact retrieval, hash verification, metric extraction, cleaning, normalization, and report generation;
- **On-chain:** submission registration, compact metric-record storage, evidence-hash storage, permission checks, and events;
- **Not in Phase I:** final metric selection, contract implementation, and M5 maintenance-responsiveness implementation.

7.2 Metric-Agnostic Record Shape

The future contract interface can remain metric-agnostic until Phase II selects a metric. A minimal record shape is shown in Table 8; field names are implementation placeholders, not final Solidity declarations.

Record	Core fields	Role
`ProjectSubmission`	`projectId`, `developerId`, `versionId`, `archiveLink`, `archiveHash`, `tags`, `storagePersistence`, `submittedAt`	Registers the artifact/provenance claim that later metric evidence depends on.
`MetricRecord`	`projectId`, `versionId`, `metricName`, `metricValue`, `evidenceHash`, `analysisPolicy`, `recordedAt`	Stores the compact selected metric result and binds it to an auditable off-chain report.

Table 8: Metric-agnostic record shape for the future DESTINO PoC.

The `analysisPolicy` field is included because the current report now makes population and sensitivity rules explicit: human-only M3/M4 outputs, short-window flags, and bot-sensitivity summaries are not incidental cleaning details. They are part of the evidence interpretation that must accompany any future metric record.

7.3 Transaction and Replay Flow

The intended end-to-end flow is:

1. a developer registers signed project/archive metadata and an archive hash;
2. an off-chain verifier retrieves the artifact and checks hash equality;
3. the selected Phase II metric is computed using the pinned script version and declared analysis policy;
4. a compact metric value and evidence-report hash are recorded on-chain;
5. an event is emitted so an indexing/search component can expose the new datapoint;
6. auditors can replay the same computation from the artifact hash, script version, window definition, and evidence report.

This design preserves the core DESTINO requirement: trust is not based on a private metric calculation alone, but on a replayable chain of artifact commitments, deterministic scripts, and compact public records.

7.4 How Phase I Outputs Support the Blueprint

The current Phase I outputs are already shaped for this future integration:

- frozen projects, versions, and windows provide stable input identifiers;
- clean and human-only metric tables provide candidate values that can be reduced to compact records;
- duration flags and bot summaries document analysis policies that affect interpretation;
- logs and reproducibility checkpoints provide the replay trail for third-party verification;
- the feasibility rubric defines the gate that prevents an empirically attractive but contract-hostile metric from becoming the PoC.

This means the current report completes the DESTINO alignment at the planning and evidence level. The implementation step remains properly deferred until Phase II identifies the metric with the best empirical and feasibility tradeoff.

8 Phase I: Dataset Freeze and Version-Window Definition

This section documents the completed Phase I freeze workflow for the centralized dataset: repository metadata freeze, stable-tag freeze, and consecutive-window construction. The first four repositories were used as a pilot to validate the workflow. The final Phase I dataset keeps those four repositories and adds six more projects from different ecosystems. The cohort labels are retained only as provenance; the analysis dataset is centralized.

8.1 Repository Freeze

The centralized Phase I dataset contains ten mature GitHub-hosted repositories. Storage persistence is recorded as `High (GitHub)` for every project. Software-type tags are kept short and descriptive so that later analysis can group projects without losing the identity of each repository.

Cohort	Project	Repository URL	Software-type tags
Pilot	flask	`https://github.com/pallets/flask`	web-framework, library
Pilot	requests	`https://github.com/psf/requests`	http-client, library
Pilot	urllib3	`https://github.com/urllib3/urllib3`	http-client, library
Pilot	click	`https://github.com/pallets/click`	cli-toolkit, library
Extended	brew	`https://github.com/Homebrew/brew`	package-manager, cli-tool, ecosystem-infrastructure
Extended	curl	`https://github.com/curl/curl`	network-transfer-tool, cli-tool, library
Extended	gin	`https://github.com/gin-gonic/gin`	web-framework, http-framework, library
Extended	junit5	`https://github.com/junit-team/junit5`	testing-framework, library
Extended	prettier	`https://github.com/prettier/prettier`	code-formatter, cli-tool, developer-tool
Extended	tokio	`https://github.com/tokio-rs/tokio`	async-runtime, concurrency-library, library

Table 9: Centralized Phase I repository freeze.

8.2 Tag Freeze and Consecutive Windows

For each project, eight stable release tags are frozen. Let $\{T_1, \ldots, T_8\}$ be the frozen tags sorted chronologically from oldest to newest. Seven windows are then defined as $W_k = (T_k \rightarrow T_{k+1})$ for $k \in \{1, \ldots, 7\}$. Each frozen tag is bound to a specific commit hash, including dereferencing annotated

tags to their target commits. This gives each metric value a replayable repository state and a deterministic release interval.

The final freeze is captured in `data/dataset_freeze_all.csv`, `data/versions_all.csv`, and `data/windows_all.csv`. The earlier pilot-only CSVs were consolidated into these centralized files and removed from the active evidence layer to avoid maintaining parallel copies of the same dataset.

Centralized Phase I totals are:

- 10 repositories;
- 80 frozen tags (10 projects × 8 tags);
- 70 consecutive windows (10 projects × 7 windows);
- tag-date coverage from 2022-06-06 to 2026-04-29.

Project	Tags	Windows	First tag	Last tag
brew	8	7	5.1.1	5.1.8
click	8	7	8.1.6	8.3.1
curl	8	7	curl-8_14_0	curl-8_20_0
flask	8	7	3.0.0	3.1.3
gin	8	7	v1.8.1	v1.12.0
junit5	8	7	r6.0.0	r5.14.4
prettier	8	7	3.7.1	3.8.3
requests	8	7	v2.30.0	v2.32.5
tokio	8	7	tokio-1.47.3	tokio-1.52.1
urllib3	8	7	2.2.3	2.6.3

Table 10: Centralized version coverage by project.

Project	Stable tag rule	Reason for project-specific rule
brew	`^\d +\.\d +\.\d +$`	Plain semantic-version release tags.
click	`^\d +\.\d +\.\d +$`	Plain semantic-version release tags.
curl	`^curl-\d +_\d +_\d +$`	Stable curl releases use a `curl-` prefix and underscores; release candidates are excluded.
flask	`^\d +\.\d +\.\d +$`	Plain semantic-version release tags.
gin	`^v\d +\.\d +\.\d +$`	Stable Go module tags use a `v` prefix.
junit5	`^r\d +\.\d +\.\d +$`	Stable JUnit tags use an `r` prefix; milestone and release-candidate tags are excluded.
prettier	`^\d +\.\d +\.\d +$`	Stable releases use plain semantic versions; alpha and non-version tags are excluded.
requests	`^v\d +\.\d +\.\d +$`	Stable release tags use a `v` prefix.
tokio	`^tokio-\d +\.\d +\.\d +$`	The repository contains crate-specific tags; only main Tokio runtime releases are included.
urllib3	`^\d +\.\d +\.\d +$`	Plain semantic-version release tags.

Table 11: Stable-tag rules for the centralized repositories.

Project	Min window days	Max window days	Mean window days
brew	0.83	8.96	4.99
click	10.04	489.99	121.58
curl	7.00	63.00	48.00
flask	16.05	219.96	124.66
gin	60.82	378.26	194.70
junit5	0.03	69.91	29.73
prettier	0.87	78.36	19.79
requests	0.26	376.04	119.74
tokio	0.58	58.99	14.87
urllib3	3.00	170.05	68.89

Table 12: Window-duration statistics for the centralized release windows.

Two details need to be kept in view during Phase II. First, some repositories release several stable tags very close together, especially `junit5`, `brew`, `prettier`, and `tokio`. Second, JUnit has overlapping stable lines in the frozen interval. This is not corrected away: the freeze follows the declared rule of latest stable repository tags, and the short-window sensitivity outputs document the effect of the resulting release cadence.

9 Phase I: Metric Extraction Pipeline

Candidate metrics M1–M4 are computed off-chain from Git history and frozen tag windows. The scripts are deterministic command-line programs: they consume declared CSV inputs and write CSV outputs without manual editing. The centralized execution consumes `data/dataset_freeze_all.csv`, `data/versions_all.csv`, and `data/windows_all.csv`. Pilot-specific files remain in the workspace for traceability, but the suffixed `_all` artifacts are the final Phase I evidence base.

9.1 Primary CSV outputs

- `out/loc_per_tag_all.csv`: per (project, tag) snapshot size with total, blank, comment, and code lines.
- `out/metrics_window_dev_all.csv`: per (project, window, developer) commits, added/deleted lines, churn, binary-file count, activity days, and first/last committer timestamps.
- `out/metrics_window_totals_all.csv`: per (project, window) totals and unique-author counts.
- `out/metrics_window_dev_clean_all.csv` and `out/metrics_window_totals_clean_all.csv`: validated clean datasets used as the all-account audit baseline.
- `out/ownership_dev_shares_clean_all.csv` and `out/ownership_summary_clean_all.csv`: all-account ownership and concentration computed strictly from clean data.
- `out/metrics_window_dev_human_all.csv`, `out/metrics_window_totals_human_all.csv`, and `out/ownership_summary_human_all.csv`: human-only M3/M4 analysis population.
- `out/metrics_window_dev_human_normalized_all.csv` and `out/metrics_window_totals_human_normalized_all.csv`: duration-normalized human-only metrics.
- `out/window_duration_flags_all.csv` and `out/window_duration_sensitivity_summary_all.csv`: short-window flags and sensitivity thresholds.
- `out/bot_summary_by_project_all.csv`, `out/bot_summary_by_window_all.csv`, and `out/bot_identity_summary_all.csv`: automation impact summaries.

9.2 Script-to-output mapping

Script	Primary role in the centralized run
scripts/extract_window_metrics.py	Extracts M2/M3 base data from Git revision ranges and flags known automated identities.
scripts/compute_loc_per_tag.py	Legacy Python-only M1 extractor retained for audit continuity with the pilot run.
scripts/compute_loc_per_tag_multilang.py	Computes multi-language M1 snapshots for the expanded dataset using tracked source files.
scripts/clean_round1_metrics.py	Canonicalizes developer-window rows by (project_id, window_id, author_email) and recomputes clean totals.
scripts/recompute_ownership_from_clean.py	Computes clean ownership shares, top-1/top-3 concentration, and Gini values.
scripts/compute_bot_sensitivity_outputs.py	Exports human-only tables and bot-sensitivity summaries while keeping all-account outputs for audit.
scripts/compute_duration_normalized_metrics.py	Computes per-day and per-KLOC-day fields and marks short windows as sensitivity_only.

Table 13: Executable mapping from Phase I scripts to centralized outputs.

9.3 Metric definitions (implementation-level)

M1 (LOC/SLOC). For each frozen tag commit, source files are counted and classified into blank, comment, and code lines. The centralized run uses a multi-language policy documented in data/loc_language_policy_all.csv; it counts tracked source files for the main languages of the ten repositories and excludes build, cache, dependency, and version-control directories. This keeps M1 comparable as a source-size indicator while avoiding generated or vendored material where possible.

M2 (Churn). For each window $(from\_tag \rightarrow to\_tag)$, Git numstat output is used to sum lines added and deleted per developer. Churn is defined as added + deleted. Binary file changes are not assigned line counts, but they are retained through binary_file_count.

M3 (Cadence). Cadence is represented by commit counts, activity days, and first/last committer timestamps per developer-window. Human-only cadence outputs are the primary developer-behavior population. All-account cadence outputs are retained as an audit baseline because they describe total repository activity, including automated updates.

M4 (Ownership and concentration). For each project-window, developer shares are computed as:

$$share_{d,w}^{(churn)} = \frac{churn_{d,w}}{\sum_{d'} churn_{d',w}}, \quad share_{d,w}^{(commits)} = \frac{commits_{d,w}}{\sum_{d'} commits_{d',w}}.$$

The pipeline derives top-1/top-3 shares and Gini concentration values. Because M4 is intended to describe human maintainership concentration, the primary M4 outputs are the human-only tables. All-account M4 remains useful for audit and sensitivity analysis.

Duration-normalized derived metrics. Release windows vary from less than one hour to more than one year. The pipeline therefore derives commits_per_day, churn_per_day, and churn_per_kloc_day. Windows shorter than seven days are retained for traceability but marked as sensitivity_only. Phase II should not let these windows drive primary cadence conclusions.

Bot-sensitivity derived metrics. Automated accounts are explicitly labeled through is_bot. The final centralized dataset flags six automation identities: dependabot[bot], renovate[bot], pre-commit-ci[bot], pre-commit-ci-lite[bot], autofix-ci[bot], and BrewTestBot. The bot-sensitivity outputs quantify their contribution and export a parallel human-only population for M3 and M4.

9.4 Error handling and traceability

The extraction pipeline is fault-tolerant at repository-window granularity:

- Git command failures are appended to the corresponding `logs/*.log` file and processing continues where possible;
- cleaning, ownership, duration-normalization, and bot-sensitivity scripts write structured logs with row counts and validation outcomes;
- all derived tables are built from declared input files and script rules, not from manual spreadsheet edits.

This gives the report a reproducible audit trail: a reader can follow the dataset from frozen repositories and commits to clean, human-only, normalized metric tables.

10 Phase I: Data Integrity Verification and Cleaning

10.1 Sanity checks

The final Phase I dataset is checked at three levels:

- window integrity: each of the ten projects has seven consecutive windows from eight frozen tags;
- row arithmetic: for every developer-window row, `churn` = `added` + `deleted`;
- aggregation consistency: window totals equal the sums of the corresponding developer rows.

All 70 centralized windows pass these checks. No clean window has zero commits or zero churn. The all-account clean dataset contains 1,118 developer-window rows, and the centralized window-total table contains 70 rows.

10.2 Canonical identity

Developer identity is canonicalized during extraction using the key:

$$(\texttt{project\_id}, \texttt{window\_id}, \texttt{author\_email})$$

For each key, commits are aggregated by email address and a canonical `author_name` is selected using the most frequent non-`unknown` display name. The cleaning pass found no duplicate key groups in either the extended cohort or the centralized dataset:

- extended dataset: 845 input rows, 845 clean rows, 0 duplicate groups merged;
- centralized dataset: 1,118 input rows, 1,118 clean rows, 0 duplicate groups merged.

10.3 Bot labeling

Automated accounts are retained in the clean all-account dataset and marked with `is_bot`. This makes the automation policy visible and allows two populations to be reported:

- all-account outputs, used as the audit baseline for total repository activity;
- human-only outputs, used as the primary population for M3 cadence and M4 ownership/concentration.

The centralized clean dataset contains 55 bot developer-window rows and 1,063 human developer-window rows. Bot accounts contribute 999 of 9,560 commits (10.45%) but only 9,294 of 2,066,059 churn lines (0.45%). This confirms that automation affects commit-frequency and ownership/cadence interpretation more than aggregate churn magnitude.

Project	Bot rows	Bot commits	Bot commit %	Bot churn %
brew	10	67	17.72	9.10
click	8	55	13.03	0.94
curl	10	127	3.78	0.33
flask	2	24	13.33	1.70
gin	5	100	25.32	5.15
junit5	7	556	14.10	0.12
prettier	4	5	6.17	0.94
requests	3	27	17.76	4.94
tokio	1	1	0.20	0.00
urllib3	5	37	25.00	2.19

Table 14: Project-level automation contribution from `out/bot_summary_by_project_all.csv`.

10.4 Known limitation: timestamp semantics

Some commits included in a tag-to-tag revision range may have committer timestamps outside the tag-date interval. This can happen when work was committed on a branch before it became reachable from a later tagged release. Churn, commit counts, and ownership shares are computed from Git revision ranges and are not affected by this timestamp drift. Cadence features based on first/last commit timestamps are more sensitive, so Phase II uses human-only commit counts and duration-normalized rates as the primary cadence evidence and treats timestamp-derived active-day fields as sensitivity evidence.

10.5 Issue-resolution ledger

Issue	Observed count	Resolution in centralized Phase I dataset
Display-name variation for the same email/window	0 duplicate groups after cleaning	Canonical aggregation by email key is performed during extraction and checked during cleaning.
Incorrect unique-author counting in totals	0 mismatches	Window totals compute unique authors from canonical `author_email` values.
Bot contamination in M3/M4	55 bot rows	Bots are retained for audit, human-only M3/M4 outputs are exported, and bot contribution is summarized by project, window, and identity.
Very short release windows	22 windows below 7 days	These windows are retained but marked `sensitivity_only`; robust cadence analysis excludes them.
Timestamp drift between commit dates and tag dates	Documented limitation	Revision-range metrics remain valid; timestamp-derived cadence fields require sensitivity checks.

Table 15: Centralized Phase I issue ledger and implemented validation actions.

11 Centralized Extraction Summary (Phase I Deliverables)

The centralized Phase I extraction spans 70 windows and produces 1,118 clean all-account developer-window rows after identity canonicalization. Aggregate all-account totals are 9,560 commits and 2,066,059 churn lines. The primary human-only population for M3/M4 contains 1,063 developer-window rows, 8,561 commits, and 2,056,765 churn lines.

Project	Windows	All commits	All churn	Human commits	Human churn
brew	7	378	43,524	311	39,565
click	7	422	29,176	367	28,901
curl	7	3,356	421,663	3,229	420,253
flask	7	180	13,159	156	12,935
gin	7	395	22,079	295	20,943
junit5	7	3,942	1,456,803	3,386	1,455,032
prettier	7	81	8,194	76	8,117
requests	7	152	3,277	125	3,115
tokio	7	506	55,498	505	55,496
urllib3	7	148	12,686	111	12,408

Table 16: Centralized project-level extraction summary from clean all-account and human-only outputs.

Scope	Windows	Developer rows	Total commits	Total churn
All accounts	70	1,118	9,560	2,066,059
Human-only M3/M4 population	70	1,063	8,561	2,056,765
Automated accounts only	42	55	999	9,294

Table 17: Centralized global extraction totals from all-account, human-only, and bot-only populations.

For the automated-account row, the window count reports windows with at least one bot row.

Project	First code lines	Last code lines	Delta	Mean code lines
brew	213,073	220,989	7,916	217,077
click	13,454	16,504	3,050	14,691
curl	224,573	225,738	1,165	222,721
flask	12,586	12,977	391	12,816
gin	13,510	17,974	4,464	14,730
junit5	132,705	132,689	-16	132,821
prettier	112,668	113,217	549	113,035
requests	7,964	8,412	448	8,257
tokio	86,219	95,257	9,038	92,097
urllib3	23,694	25,260	1,566	24,748

Table 18: M1 snapshot-size statistics by project.

The expanded dataset is intentionally heterogeneous. Large projects such as `junit5`, `curl`, and `brew` dominate absolute churn and line-count totals, while smaller libraries such as `requests`, `flask`, and `click` provide a continuity check against the pilot workflow. For this reason, Phase II should rely on normalized and within-project views when comparing candidate metric behavior.

12 Detailed Descriptive Results for M1–M4

This section reports the descriptive outputs computed from the centralized Phase I dataset. The tables are not used to rank metrics or select a proof-of-concept metric. They document the empirical base that Phase II will compare under the predefined criteria.

12.1 M1 Snapshot size evolution

Project	First code	Last code	Delta	Delta %	Min	Max
brew	213073	220989	7916	3.72	213073	220989
click	13454	16504	3050	22.67	13454	16504
curl	224573	225738	1165	0.52	219978	225738
flask	12586	12977	391	3.11	12586	12977
gin	13510	17974	4464	33.04	13510	17974
junit5	132705	132689	-16	-0.01	132615	133150
prettier	112668	113217	549	0.49	112668	113217
requests	7964	8412	448	5.63	7964	8425
tokio	86219	95257	9038	10.48	86219	95385
urllib3	23694	25260	1566	6.61	23694	25260

Table 19: M1 code-line evolution across frozen tags. First and last values follow chronological tag order.

M1 captures source-size change over the frozen release interval. The largest relative growth appears in `gin`, `click`, and `tokio`. `junit5` is effectively stable in total code-line size across its selected tags, even though its windows contain substantial churn.

12.2 M2 churn and M3 cadence intensity

Project	Human commits	Human churn	Churn/commit	Commits/day	Churn/day	Mean authors/window
brew	311	39565	127.22	8.904	1132.80	11.29
click	367	28901	78.75	0.431	33.96	14.29
curl	3229	420253	130.15	9.610	1250.74	27.57
flask	156	12935	82.92	0.179	14.82	6.43
gin	295	20943	70.99	0.216	15.37	25.29
junit5	3386	1455032	429.72	16.268	6990.58	16.14
prettier	76	8117	106.80	0.549	58.58	2.43
requests	125	3115	24.92	0.149	3.72	8.43
tokio	505	55496	109.89	4.853	533.33	32.71
urllib3	111	12408	111.78	0.230	25.73	7.29

Table 20: M2/M3 human-only project-level intensity.

The centralized dataset has a wider activity range than the pilot. `curl` and `junit5` carry high commit and churn volume, while `requests`, `flask`, and `urllib3` provide lower-volume contrast. These differences are useful for Phase II because a candidate metric should remain interpretable across both high- and low-activity projects.

12.3 Bot-sensitivity outputs

Project	Bot rows	Bot commit %	Bot churn %	Windows with bots	Bot top-commit windows
brew	10	17.72	9.10	5	0
click	8	13.03	0.94	6	1
curl	10	3.78	0.33	6	0
flask	2	13.33	1.70	1	0
gin	5	25.32	5.15	5	5
junit5	7	14.10	0.12	7	0
prettier	4	6.17	0.94	3	0
requests	3	17.76	4.94	3	2
tokio	1	0.20	0.00	1	0
urllib3	5	25.00	2.19	5	1

Table 21: Automation impact summary. Percentages are relative to all-account project totals.

Across the centralized dataset, bot rows contribute 999 of 9560 commits (10.45%) but only 9294 of 2,066,059 churn lines (0.45%). Automation therefore matters most for cadence and ownership interpretation, not for aggregate churn volume. The primary M3 and M4 evidence uses human-only outputs, while all-account outputs remain available for audit and sensitivity checks.

12.4 Duration-normalized analysis outputs

Sensitivity threshold	Window count	Affected windows
<1 day	9	requests_W03; requests_W04; brew_W02; junit5_W02; junit5_W04; junit5_W06; prettier_W02; tokio_W01; tokio_W04
<7 days	22	requests_W03; requests_W04; urllib3_W05; urllib3_W06; brew_W02; brew_W03; brew_W04; brew_W05; brew_W06; junit5_W02; junit5_W04; junit5_W06; prettier_W01; prettier_W02; prettier_W03; prettier_W05; prettier_W07; tokio_W01; tokio_W04; tokio_W05; tokio_W06; tokio_W07
<14 days	27	The previous windows plus click_W04, requests_W05, brew_W01, brew_W07, and curl_W01.

Table 22: Short-window sensitivity flags. Windows below seven days are retained but marked as sensitivity_only.

Window	Duration days	Commits/day	Churn/day	Analysis set
junit5_W06	0.030	26909.313	10006801.83	sensitivity_only
junit5_W04	0.034	23433.962	8822406.47	sensitivity_only
junit5_W02	0.135	4976.771	375831.47	sensitivity_only
tokio_W01	0.577	277.240	37366.70	sensitivity_only
tokio_W04	0.854	291.516	31412.33	sensitivity_only
junit5_W05	39.870	7.399	6573.49	primary

Table 23: Highest human-only churn/day windows. Very short windows can dominate per-day rates, so they are not used as primary cadence evidence.

The normalized artifacts provide aggregate per-day commit and churn rates, per-KLOC-day churn rates, and parallel developer-window fields. Raw totals remain useful descriptive evidence, while duration-sensitive comparisons should use the normalized tables together with the analysis_set flag.

12.5 M3 developer activity-day profile

Project	Mean active days/dev-window	Min	Max	Min window days	Max window days
brew	1.90	1	7	0.83	8.96
click	2.10	1	19	10.04	489.99
curl	5.04	1	58	7.00	63.00
flask	1.84	1	17	16.05	219.96
gin	1.50	1	12	60.82	378.26
junit5	6.74	1	95	0.03	69.91
prettier	1.59	1	4	0.87	78.36
requests	1.54	1	8	0.26	376.04
tokio	1.92	1	25	0.58	58.99
urllib3	1.75	1	13	3.00	170.05

Table 24: M3 human-only activity-day profile.

The activity-day profile confirms that cadence is sensitive to both project rhythm and release-window construction. This is why Phase II should compare cadence-related indicators with both full-data and robust, short-window-filtered views.

12.6 M4 ownership and concentration profile

Project	Top1 commits	Top3 commits	Top1 churn	Top3 churn	Gini commits	Gini churn
brew	0.35	0.68	0.64	0.86	0.40	0.71
click	0.36	0.69	0.50	0.86	0.51	0.72
curl	0.46	0.88	0.54	0.97	0.81	0.89
flask	0.71	0.89	0.89	0.99	0.49	0.68
gin	0.33	0.52	0.48	0.73	0.23	0.57
junit5	0.79	0.96	0.90	0.98	0.87	0.91
prettier	0.87	0.96	0.83	0.99	0.35	0.32
requests	0.57	0.79	0.71	0.94	0.22	0.41
tokio	0.30	0.61	0.42	0.71	0.31	0.56
urllib3	0.63	0.78	0.69	0.94	0.31	0.50

Table 25: Mean human-only M4 concentration indicators.

M4 shows substantial variation in human concentration. Some projects have broad participation by author count but still concentrate churn in a small set of contributors. This distinction is important for Phase II because ownership-style metrics may reward sustained maintainership differently from simple activity-volume metrics.

13 Phase I Execution Ledger

This section records the operational sequence executed for the centralized Phase I dataset. The ledger links each step to its primary artifact and gives the validation result that makes the artifact usable in later analysis.

Step	Action	Primary artifact(s)	Verification result
2.1	Freeze repository metadata	data/dataset_freeze_all.csv	10 repositories frozen with cohort labels, repository URLs, persistence status, and software-type tags.
2.2	Freeze stable release tags	data/versions_all.csv; data/version_tag_rules_all.csv	80 stable tags frozen, eight per repository, with each tag tied to an immutable commit hash.
2.3	Build consecutive windows	data/windows_all.csv	70 tag-to-tag windows generated in chronological order with no chain or chronology errors.
2.4.1	Extract raw developer-window metrics	out/metrics_window_dev_all.csv; out/metrics_window_totals_all.csv	1118 developer-window rows and 70 window-total rows produced; extraction error logs remained empty.
2.4.2	Clean and validate developer rows	out/metrics_window_dev_clean_all.csv; out/metrics_window_totals_clean_all.csv; logs/clean_all_metrics.log	Clean rows remained 1118; duplicate groups merged were 0; zero-commit and zero-churn windows were 0.
2.4.3	Compute LOC/SLOC snapshots (M1)	out/loc_per_tag_all.csv; data/loc_language_policy_all.csv; logs/loc_full_recompute_20260508.log	80 per-tag rows produced using the centralized multi-language source policy; full recomputation produced 80 rows with 0 mismatches.
2.4.4	Compute ownership/concentration (M4)	out/ownership_dev_shares_clean_all.csv; out/ownership_summary_clean_all.csv	1118 all-account developer-share rows and 70 summary rows generated from clean data.
2.4.5	Separate automation and human-only evidence	out/metrics_window_dev_human_all.csv; out/ownership_summary_human_all.csv; out/bot_summary_by_project_all.csv	1063 human developer-window rows, 70 human ownership summaries, and automation summaries for 6 bot identities generated.
2.4.6	Normalize for uneven window duration	out/metrics_window_totals_human_normalized_all.csv; out/metrics_window_dev_human_normalized_all.csv; out/window_duration_flags_all.csv	Per-day and per-KLOC-day fields generated; 22 windows below seven days marked as sensitivity_only.
2.4.7	Validate canonical repository layout	logs/canonical_repos_validation.log	10 colocated repositories checked; 80 frozen tags resolved to their frozen commit hashes; 0 missing or mismatched tags/commits.
2.4.8	Remove superseded pilot-only files	logs/pilot_cleanup_20260505.log	32 pilot-only CSV/log files removed after verification that the active evidence is centralized in the _all artifacts.
2.4.9	Remove superseded extended/scoped files	logs/centralized_cleanup_20260508.log	42 extended CSV/log files and 8 duplicate scoped output folders removed after subset and rerun verification against the centralized _all artifacts.

Table 26: Execution ledger for the centralized Phase I pipeline.

13.1 Cleaning and validation outcomes

The cleaning and recomputation logs provide concrete numerical guarantees:

- input developer-window rows: 1118; cleaned rows: 1118;
- duplicate key groups merged during cleaning: 0;
- windows with zero churn: 0; windows with zero commits: 0;
- bad windows for share-sum checks: 0;
- bad windows for unique-author consistency: 0;
- human-only rows generated for primary M3/M4 analysis: 1063;
- bot-flagged rows retained for audit: 55;
- bot contribution: 999 commits (10.45%) and 9294 churn lines (0.45%);
- duration-normalized rows generated: 70 aggregate rows and 1118 developer rows for the all-account layer, plus 70 aggregate rows and 1063 developer rows for the human-only layer;

- windows shorter than seven days and flagged as `sensitivity_only`: 22.

These outcomes confirm that the centralized Phase I artifacts are internally consistent and ready to serve as auditable input for Phase II.

14 Reproducibility Protocol and Data Lineage

14.1 Artifact layout

Phase I artifacts are organized under a single workspace root, `Data,Outputs,Logs,Scripts/Phase 1/`, with fixed subdirectories: `data/`, `repos/`, `scripts/`, `out/`, and `logs/`. The final evidence layer uses the `_all` suffix. Pilot-only and extended-only CSVs have been consolidated into the centralized files and removed from the active evidence layer. The centralized `_all` files are the inputs for the next phase.

All ten Git working copies are now colocated under the canonical `repos/` directory. Each repository uses the official GitHub URL from `data/dataset_freeze_all.csv` as its `origin`. The layout has also been validated against `data/versions_all.csv`: all 80 frozen tags resolve to the frozen commit hashes in the centralized repository folder. The validation record is stored in `logs/canonical_repos_validation.log`.

14.2 Execution protocol

The pipeline is deterministic given the frozen version and window files. A full rerun follows this order:

1. freeze repository metadata in `data/dataset_freeze_all.csv`;
2. freeze stable tags and commit hashes in `data/versions_all.csv`;
3. build consecutive tag-to-tag windows in `data/windows_all.csv`;
4. run `scripts/extract_window_metrics.py` to generate developer-window and window-total rows;
5. run `scripts/clean_round1_metrics.py` to canonicalize developer rows and recompute clean totals;
6. run `scripts/compute_loc_per_tag_multilang.py` for M1 snapshot-size evidence;
7. run `scripts/recompute_ownership_from_clean.py` for M4 concentration evidence;
8. run `scripts/compute_bot_sensitivity_outputs.py` to export human-only and bot-sensitivity tables;
9. run `scripts/compute_duration_normalized_metrics.py` for per-day, per-KLOC-day, and short-window flags;
10. verify row counts and validation logs before using the outputs in Phase II.

14.3 Lineage checkpoints

Artifact	Rows	Role
data/dataset_freeze_all.csv	10	Repository metadata and cohort membership.
data/version_tag_rules_all.csv	10	Stable-tag selection rules for all repositories.
data/loc_language_policy_all.csv	10	Source-extension policy for M1 LOC/SLOC counting.
data/versions_all.csv	80	Frozen tags and commit hashes (10 projects × 8 tags).
data/windows_all.csv	70	Consecutive windows (10 projects × 7 windows).
out/metrics_window_dev_clean_all.csv	1118	Validated all-account developer-window dataset.
out/metrics_window_totals_clean_all.csv	70	All-account window totals from clean developer rows.
out/metrics_window_dev_human_all.csv	1063	Human-only developer-window dataset for primary M3/M4 analysis.
out/metrics_window_totals_human_all.csv	70	Human-only window totals.
out/loc_per_tag_all.csv	80	M1 snapshot-size values per frozen tag.
out/ownership_dev_shares_human_all.csv	1063	Human-only ownership shares.
out/ownership_summary_human_all.csv	70	Human-only concentration statistics for M4.
out/metrics_window_totals_human_normalized_all.csv	70	Human-only duration-normalized aggregate metrics.
out/metrics_window_dev_human_normalized_all.csv	1063	Human-only duration-normalized developer metrics.
out/window_duration_flags_all.csv	70	Short-window flags and primary/sensitivity labels.
out/bot_summary_by_project_all.csv	10	Project-level automation contribution summary.
out/bot_summary_by_window_all.csv	70	Window-level automation contribution and top-contributor flags.
out/bot_identity_summary_all.csv	6	Automation identity summary.
logs/canonical_repos_validation.log	1	Canonical repository-layout validation log.
logs/loc_full_recompute_20260508.log	1	Full M1 LOC recomputation audit log; recomputed rows matched out/loc_per_tag_all.csv exactly.
logs/pilot_cleanup_20260505.log	1	Cleanup record for pilot-only files removed after consolidation.
logs/centralized_cleanup_20260508.log	1	Cleanup and verification record for extended-only and scoped duplicate files removed after consolidation.

Table 27: Centralized Phase I data lineage checkpoints used for reproducibility and auditing.

14.4 Automated validation evidence

Validation evidence is recorded in log files and confirms that:

- duplicate-key groups merged in cleaning: 0;
- windows with zero total churn: 0;
- windows with zero total commits: 0;
- windows failing share-sum validations: 0;
- windows failing unique-author consistency checks: 0;
- all-account total mismatches found during bot-sensitivity validation: 0;
- windows shorter than seven days and flagged as sensitivity_only: 22;
- bot rows retained for audit: 55; human-only rows exported for primary M3/M4 analysis: 1063.
- canonical repository-layout validation: 10 repositories checked, 80 frozen tag rows checked, 0 missing or mismatched commits, and 0 missing or mismatched tags.
- pilot-only cleanup: 32 superseded pilot CSV/log files removed after their evidence was consolidated into the centralized _all layer.
- extended/scoped cleanup: 42 superseded extended CSV/log files and 8 duplicate scoped output folders removed after verification against the centralized _all layer.

These checks establish that downstream Phase II analysis will operate on a coherent and fully auditable dataset.

15 Threats to Validity

15.1 Construct validity

M2 uses churn as a proxy for change magnitude, which does not directly encode semantic complexity, defect risk, or architectural importance. M3 cadence features are sensitive to timestamp semantics; commit-date and tag-date boundaries are therefore treated as robustness concerns rather than ignored. Automated accounts can inflate commit-frequency and ownership/cadence interpretations, so M3 and M4 use human-only outputs as the primary population and keep all-account outputs for sensitivity checks. To reduce construct ambiguity, the study keeps raw fields such as added lines, deleted lines, commits, and activity days alongside derived indicators.

15.2 Internal validity

Potential extraction errors were mitigated through deterministic windows, immutable commit hashes, explicit cleaning rules, and arithmetic checks such as `churn` = `added` + `deleted`. Developer identity is canonicalized by email during extraction and cleaning, window totals are recomputed from clean developer rows, and ownership summaries are regenerated from those same clean rows. Bot-sensitivity outputs also validate that all-account totals remain consistent before deriving human-only tables.

15.3 External validity

The final Phase I dataset is broader than the pilot because it covers ten repositories across Python, Ruby, C/C++, Go, Java/Kotlin, JavaScript/TypeScript, and Rust ecosystems. This improves transferability compared with the four-repository pilot, but it is still a curated open-source sample rather than a statistically representative sample of all software projects. Phase II should therefore report conclusions as evidence from this selected dataset, not as universal claims about every repository or ecosystem.

15.4 Conclusion validity

No inferential claims are made in Phase I. The current report validates the dataset, pipeline, and descriptive evidence only. Comparative conclusions are deferred to Phase II, where stability, interpretability, gaming resistance, long-term suitability, bot sensitivity, and short-window robustness will be evaluated together before any proof-of-concept metric is selected.

16 Phase I Conclusion

Phase I is complete at the centralized dataset and pipeline level. The thesis now has:

- a frozen 10-repository dataset with cohort labels and repository metadata;
- 80 frozen stable tags tied to immutable commit hashes;
- 70 consecutive release windows for tag-to-tag measurement;
- extracted and cleaned developer-window metrics for M1–M4;
- ownership and concentration outputs computed strictly from canonical clean data;
- human-only M3/M4 outputs and bot-sensitivity summaries;
- duration-normalized outputs and short-window sensitivity flags for uneven release windows;
- validation logs showing arithmetic, identity, aggregation, and ownership consistency;
- a DESTINO-aligned evidence boundary and future PoC integration blueprint.

At this stage, the work remains intentionally descriptive and methodological. No PoC metric is selected here. Selection is deferred to Phase II after comparative evaluation under the criteria in Section 4 and feasibility scoring in Section 6.1.

17 Phase II Step 1: C1 Stability Analysis

Month 3 starts with the most empirical comparison criterion: stability. The purpose of this step is to measure how much each candidate signal changes across release windows and whether developer rankings remain similar from one window to the next. This step does not select the PoC metric. It produces the first evidence layer used later together with interpretability, gaming resistance, long-term suitability, and smart-contract feasibility.

17.1 Analysis inputs and generated artifacts

The C1 script reads only the centralized Phase I `_all` evidence. It writes derived Phase II artifacts under `Data,Outputs,Logs,Scripts/Phase2/`, keeping comparative-analysis outputs separate from the frozen dataset and extraction layer.

Artifact	Rows	Role
out/c1_metric_window_values_all.csv	350	Long-format window-level values for five stability signals across 70 windows.
out/c1_metric_stability_by_project_all.csv	50	Per-project volatility statistics for each stability signal.
out/c1_metric_stability_overall_all.csv	5	Cross-project summary of window-level volatility.
out/c1_rank_input_values_all.csv	3189	Developer-level values used for adjacent-window rank comparison.
out/c1_rank_stability_pairs_all.csv	180	Adjacent-window rank correlations for M2, M3, and M4 developer-level signals.
out/c1_rank_stability_by_project_all.csv	30	Per-project rank-stability summaries.
out/c1_rank_stability_overall_all.csv	3	Cross-project rank-stability summary.
logs/c1_stability_20260508.log	1	Execution log for the C1 stability run.

Table 28: Phase II C1 stability artifacts. Paths are relative to the Phase II analysis directory.

17.2 Method

For window-level stability, five signals are compared:

- M1 endpoint KLOC, using the code-line count at each window's target tag;
- M2 churn per KLOC-day, using size- and duration-normalized churn;
- M3 commits per day, using duration-normalized activity cadence;
- M4 churn Gini, using contributor concentration in churn;
- M4 top-1 churn share, using the largest contributor's churn share.

For each project and signal, the script computes the coefficient of variation and adjacent-window relative change. The same statistics are computed twice: once on all windows, and once on the robust subset that excludes windows marked `sensitivity_only`. For developer-level rank stability, the script compares adjacent windows using Spearman rank correlation among developers who appear in both windows. Rank stability is computed for M2 developer churn/day, M3 developer commits/day, and M4 developer churn share.

17.3 Window-level stability results

Signal	Median CV	Primary CV	Primary adjacent change	Primary high-vol. projects
M1 endpoint KLOC	0.013	0.014	0.009	0
M2 churn/KLOC/day	1.208	0.890	0.706	6
M3 commits/day	0.876	0.608	0.644	4
M4 churn Gini	0.424	0.329	0.337	2
M4 top-1 churn share	0.289	0.236	0.208	1

Table 29: C1 window-level volatility summary. CV is the coefficient of variation across project windows; "primary" excludes short windows marked `sensitivity_only`.

The first result is clear: M1 is stable because project size changes slowly between adjacent stable releases. That stability makes it a useful context and normalization signal, but it does not directly measure developer contribution quality or reputation. M2 and M3 are much more dynamic. Churn is especially volatile, which is expected because release content can vary sharply even inside the same project. M4 concentration signals are more stable than raw activity measures, with top-1 churn share showing the lowest volatility among behavior-oriented signals in this pass.

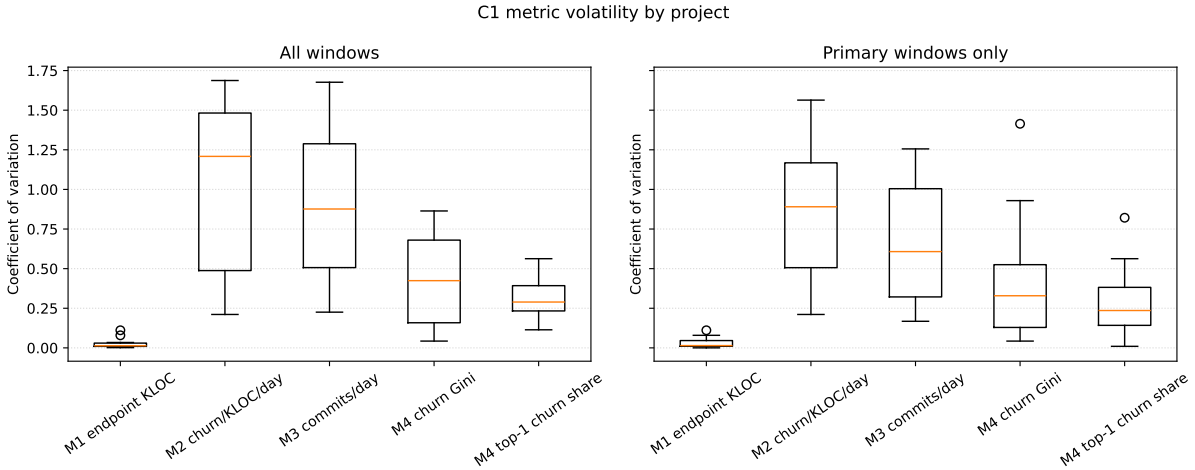

Figure 1: Distribution of per-project coefficient of variation for each C1 window-level signal.

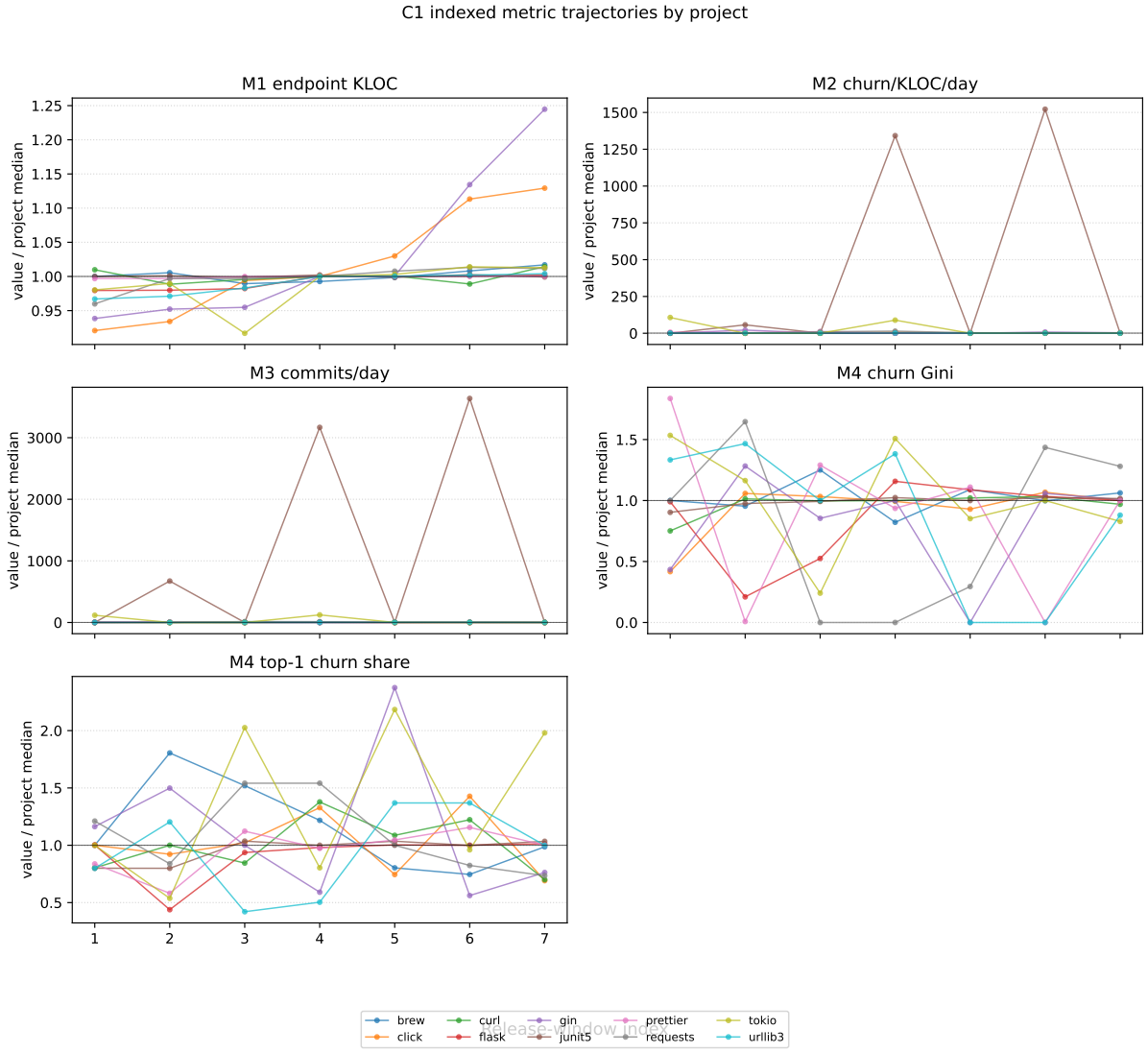

Figure 2: Indexed C1 signal trajectories by project. Each line is scaled by that project's median value for the signal.

17.4 Developer-rank stability results

Developer-level signal	Median ρ	Primary ρ	Common share	Valid pairs
M2 developer churn/day	0.668	0.608	0.172	36/60
M3 developer commits/day	0.858	0.836	0.172	33/60
M4 developer churn share	0.668	0.608	0.172	36/60

Table 30: C1 developer-rank stability summary across adjacent windows. Correlations use only developers appearing in both adjacent windows.

The rank-stability result should be read together with the common-developer share. The median overlap between adjacent windows is only 0.172, so many adjacent windows involve a partially different contributor set. Among developers who do recur, M3 commit cadence has the strongest rank continuity. M2 churn/day and M4 churn share produce the same rank-stability values because churn share is a within-window normalization of churn; it changes the unit and interpretation, but not the developer ordering inside the same window.

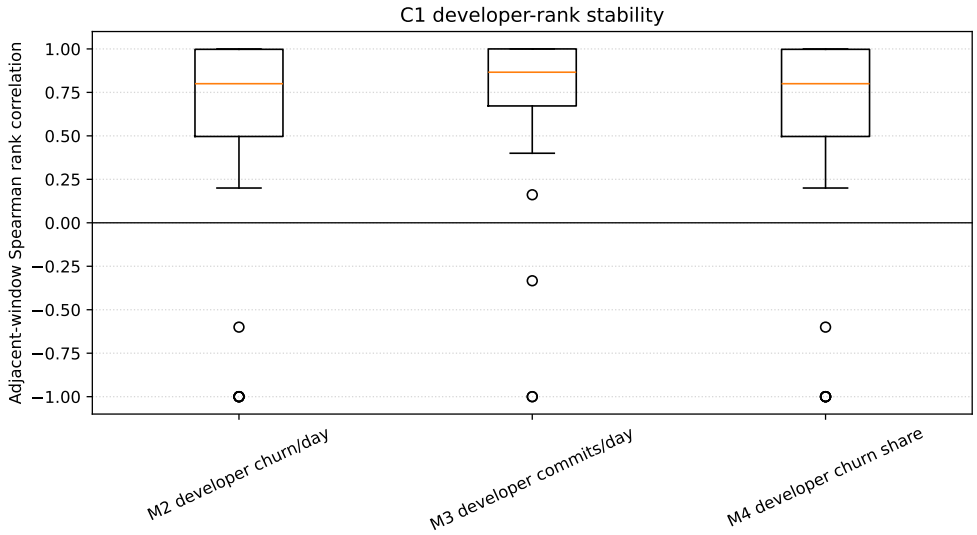

Figure 3: Adjacent-window developer-rank stability for M2, M3, and M4 developer-level signals.

17.5 C1 interpretation for the decision process

The C1 evidence does not select a final PoC metric, but it narrows the interpretation of the candidates:

- M1 is empirically stable but mainly useful as context and normalization support.
- M2 is informative about release effort but too volatile to use alone without safeguards.
- M3 has better developer-rank continuity than M2, but still depends on release cadence and contributor overlap.
- M4 concentration signals are comparatively stable and remain promising for reputation interpretation, but they require careful gaming-resistance analysis.

The next Month 3 step is therefore to evaluate C2 interpretability using the same candidate set and the C1 findings. After that, C3 gaming resistance and C4 long-term suitability can be scored before applying the smart-contract feasibility rubric.

18 Planned Next Phase: Comparative Metric Evaluation

Phase II compares the candidate metrics using the criteria defined in Section 4. The centralized Phase I artifacts are the evidence base, so this phase focuses on metric behavior rather than additional dataset construction. The first criterion, C1 stability, is reported in Section 17. Only after the remaining comparative criteria are completed will the smart-contract feasibility rubric be applied and one metric selected for the baseline proof-of-concept implementation.

18.1 Phase II execution plan

1. Complete criterion evidence for C2–C4 using the clean, human-only, bot-sensitivity, and duration-normalized `_all` outputs, building on the completed C1 stability pass.
2. Report both full-dataset results and robust results that exclude windows marked `sensitivity_only` where duration-sensitive metrics are evaluated.
3. Compare metric behavior across projects, ecosystems, and release-window lengths without selecting a PoC metric prematurely.

4. Score smart-contract feasibility dimensions for each candidate using the predefined rubric.

5. Select exactly one PoC metric using the decision protocol in Section 4.7.

6. Instantiate the selected metric inside the DESTINO-aligned metric-record blueprint in Section 7.

18.2 Decision gate criteria

The PoC metric will be selected only if it simultaneously satisfies:

- no feasibility exclusion criterion triggered under Section 6.1;
- acceptable comparative behavior under C1–C4 relative to alternatives;
- stable interpretation under bot-sensitivity and short-window robustness checks;
- reproducible extraction with no unresolved data-integrity issue.

This gate keeps implementation tied to the evidence, instead of choosing a metric because it is easy to compute or attractive in one project only.

References

[1] Weichao Gao, William G. Hatcher, and Wei Yu. A survey of blockchain: Techniques, applications, and challenges. In *Proceedings of the 27th International Conference on Computer Communication and Networks (ICCCN)*, 2018.

[2] Gerta Kapllani, Ilya Khomyakov, Ruzilya Mirgalimova, and Alberto Sillitti. An empirical analysis of the maintainability evolution of open source systems. In *Open Source Systems*. Springer, 2020.

[3] Eleni Koutrouli and Aphrodite Tsalgatidou. Reputation systems evaluation survey. *ACM Computing Surveys*, 48(3):35:1–35:28, 2015.

[4] Balraj Kumar. A survey of key factors affecting software maintainability. In *2012 International Conference on Computing Sciences*, 2012.

[5] Xiaozhou Li, Sergio Moreschini, Zheying Zhang, and Davide Taibi. Exploring factors and metrics to select open source software components for integration: An empirical study. *Journal of Systems and Software*, 188:111255, 2022.

[6] Andreas Schlosser, Marco Voss, and Lars Brueckner. Comparing and evaluating metrics for reputation systems by simulation. Conference manuscript, 2004. Local PDF copy in the "Papers to read" collection.

[7] Sharifah Mashita Syed-Mohamad, Amir Ngah, Al-Fahim Mubarak Ali, and Pantea Keikhosrokiani. Measuring software maintainability: An analysis of evolving metrics across development paradigms. *Journal of Advanced Research in Applied Sciences and Engineering Technology*, 63(2):181–195, 2026.

[8] Masoud Jamshidiyan Tehrani. Assessing vulnerability in smart contracts: The role of code complexity metrics in security analysis. arXiv preprint, 2026. arXiv:2411.17343v4.

[9] Aline Lopes Timóteo, Alexandre Álvaro, Eduardo Santana de Almeida, and Silvio Romero de Lemos Meira. Software metrics: A survey. Survey manuscript. Local PDF copy in the "Papers to read" collection.

[10] Roberto Tonelli, Giuseppe Antonio Pierro, Marco Ortu, and Giuseppe Destefanis. Smart contracts software metrics: A first study. *PLOS ONE*, 18(2):e0281043, 2023.